

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут**

**Звіт до лабораторної №1
з дисципліни «Технологія блокчейн та розподілені системи»
за темою:
Розгортання систем Ethereum та криптовалют**

Оформлення звіту:
(студент, група, рік)

Бекешева Анастасія, ФІ-51мн
Рожко Богдан, ФІ-51мн
Габунія Антон, ФІ-51мн

Мета:

Отримання навичок налаштування платформ виконання смарт- контрактів та криптовалют.

Тип роботи:

Провести порівняльний аналіз особливостей розгортання систем криптовалют у порівнянні із системою Ethereum. Зробити висновок про можливість чи неможливість взаємозаміни модулів різних систем та пояснити причини.

Актуальність:

У сучасному світі спостерігається стрімкий розвиток блокчейн-індустрії, що супроводжується появою великої кількості криптовалютних платформ із різними архітектурними підходами. Якщо на початковому етапі розвитку блокчейн-системи будувалися переважно як монолітні структури (наприклад, Bitcoin), то сьогодні відбувається поступовий перехід до модульних архітектур, які дозволяють гнучкіше налаштовувати систему, масштабувати її та інтегрувати з іншими рішеннями.

Особливо це помітно на прикладі Ethereum, який став не лише криптовалютою, а повноцінною платформою для створення децентралізованих застосунків. З іншого боку, такі системи як Litecoin, Dash та NEO демонструють альтернативні підходи до розгортання, масштабування та організації мережі.

Таким чином, дослідження особливостей розгортання цих систем і можливості взаємозаміни їхніх компонентів є актуальним завданням як з наукової, так і з практичної точки зору.

ХІД РОБОТИ

Розділ 1. Аналіз особливостей розгортання та адміністрування цільових систем

Навички розгортання блокчейн-вузлів (нод) суттєво залежать від архітектурної парадигми, яку використовує конкретна мережа.

1.1. Ethereum: Модульна архітектура та смарт-контракти

До проведення The Merge архітектура платформи Ethereum мала переважно монолітний характер. Це означало, що всі основні функції вузла — обробка транзакцій, виконання смарт-контрактів, перевірка блоків і забезпечення консенсусу — реалізовувалися в межах одного програмного клієнта. Такий підхід значно спрощував процес розгортання, оскільки адміністратору достатньо було встановити один клієнт (наприклад, Geth), синхронізувати блокчейн і підключитися до мережі. Водночас це призводило до певних обмежень, зокрема ускладнювало масштабування системи та створювало високе навантаження на один компонент.

У цей період Ethereum використовував механізм консенсусу Proof-of-Work, подібний до того, що застосовується у Bitcoin. Формування нових блоків здійснювалося майнерами, які витрачали значні обчислювальні ресурси для розв'язання криптографічних задач. Такий підхід забезпечував високий рівень безпеки, однак був енергозатратним і обмежував пропускну здатність мережі. Крім того, централізація майнінгових потужностей у великих пулах створювала потенційні ризики для децентралізації.

Після переходу до Proof-of-Stake внаслідок The Merge архітектура Ethereum зазнала суттєвих змін і стала модульною. Основною особливістю нової моделі є поділ системи на два логічно незалежні рівні: Execution Layer (рівень виконання) та Consensus Layer (рівень консенсусу). Перший відповідає за обробку транзакцій, виконання смарт-контрактів та підтримку стану мережі, тоді як другий забезпечує механізм досягнення консенсусу, тобто підтвердження і впорядкування блоків.

У результаті такого розділення процес розгортання вузла значно ускладнився. Тепер адміністратор повинен запускати два окремих клієнти, які взаємодіють між собою через спеціалізований інтерфейс (Engine API). Важливим аспектом є налаштування безпечного обміну даними між ними, зокрема із застосуванням JWT-токенів. Крім того, необхідно враховувати управління станом системи, роботу з базами даних (такими як LevelDB або Pebble) та конфігурацію мережевих параметрів.

Разом із ускладненням архітектури зросла і гнучкість системи. Модульний підхід дозволяє незалежно оновлювати або замінювати окремі компоненти, оптимізувати продуктивність і впроваджувати нові механізми масштабування, такі як

шардинг або рішення другого рівня (Layer 2). Також перехід до Proof-of-Stake суттєво знизив енергоспоживання мережі та відкрив можливості для більш ефективного розподілу ресурсів.

Після переходу на алгоритм Proof-of-Stake (The Merge), архітектура Ethereum стала дворівневою. Налаштування вузла вимагає від адміністратора розгортання двох незалежних клієнтів:

- **Execution Layer (Рівень виконання):** відповідає за обробку транзакцій та роботу віртуальної машини (EVM). Потребує налаштування RPC-інтерфейсів для взаємодії зі смарт-контрактами.
- **Consensus Layer (Рівень консенсусу):** відповідає за валідацію блоків.
Специфіка конфігурації: ключовою навичкою є налаштування безпечного з'єднання (Engine API) між двома клієнтами за допомогою системи JWT-токенів, а також управління станом (State Trie) та базою даних LevelDB/Pebble.

1.2. Bitcoin та Litecoin: Класична монолітна парадигма

Класичним прикладом монолітної архітектури в блокчейн-системах є Bitcoin та його похідні, зокрема Litecoin. У цих системах усі ключові функціональні компоненти інтегровані в межах одного програмного середовища, що реалізується у вигляді єдиного програмного демона (наприклад, *bitcoind* або *litecoind*). Такий підхід означає, що мережевий рівень (P2P-взаємодія), механізм консенсусу (Proof-of-Work), обробка транзакцій та управління даними функціонують як єдине ціле без чіткого логічного розділення на окремі модулі.

Подібна архітектура формувалася на ранніх етапах розвитку блокчейн-технологій, коли основним завданням було забезпечення надійної та безпечної передачі цифрових активів без необхідності створення складних програмованих середовищ. У результаті системи типу Bitcoin характеризуються відносною простотою реалізації, високою стабільністю та передбачуваністю поведінки. Водночас така монолітність обмежує можливості гнучкої модернізації системи, оскільки будь-які зміни потребують втручання у всю архітектуру в цілому.

З точки зору розгортання, монолітна модель значно спрощує процес налаштування вузла. Адміністрування зводиться до роботи з єдиним конфігураційним файлом формату *.conf*, у якому визначаються основні параметри функціонування системи. До ключових навичок адміністратора належать налаштування індексації транзакцій (параметр *txindex*), що дозволяє здійснювати розширений пошук по історії операцій, а також управління використанням оперативної пам'яті через параметр *dbcache*, який впливає на швидкість синхронізації та обробки даних.

Крім того, важливим аспектом є конфігурація RPC-доступу, який забезпечує взаємодію вузла з зовнішніми застосунками. Це включає налаштування мережевих портів, обмеження доступу за IP-адресами та забезпечення базового рівня безпеки

через автентифікацію. Таким чином, хоча сама система є технічно простою, ефективно адміністрування вимагає розуміння внутрішніх механізмів її роботи.

Ще однією важливою особливістю таких систем є використання моделі UTXO (Unspent Transaction Output), яка визначає принцип обліку коштів у мережі. Замість зберігання балансу рахунку система оперує набором невитрачених виходів транзакцій, що забезпечує високий рівень безпеки та прозорості. Однак ця модель суттєво обмежує можливості реалізації складної логіки, оскільки не передбачає повноцінного середовища виконання програмного коду.

У зв'язку з цим, на відміну від Ethereum, у Bitcoin та Litecoin відсутня підтримка повноцінних смарт-контрактів у класичному розумінні. Хоча існує обмежена скриптова мова для базових умовних операцій, вона не дозволяє створювати складні децентралізовані застосунки. Це підкреслює фундаментальну різницю між монолітними криптовалютами першого покоління та сучасними модульними платформами.

Отже, монолітна парадигма забезпечує простоту, надійність і безпеку, але водночас обмежує гнучкість, масштабованість та функціональність системи. Саме ці обмеження стали однією з причин переходу новіших платформ до модульних архітектур, які дозволяють розділяти функції та розвивати систему більш незалежно та ефективно.

1.3. Dash: Дворівнева топологія мережі

Криптовалютна платформа Dash є еволюційним розвитком ідей, закладених у Bitcoin, проте водночас вона впроваджує низку суттєвих архітектурних змін, спрямованих на підвищення швидкості транзакцій, ефективності управління мережею та розширення функціональних можливостей. Однією з ключових особливостей Dash є використання дворівневої топології мережі, яка відрізняє її від класичних монолітних систем.

На відміну від Bitcoin, де всі вузли виконують приблизно однакові функції, у Dash мережа поділяється на два рівні: перший рівень складається з майнерів, які відповідають за створення блоків за допомогою механізму Proof-of-Work, тоді як другий рівень представлений так званими майстернодами (Masternodes). Такий підхід дозволяє розподілити функціональність між різними типами учасників мережі та забезпечити виконання додаткових сервісів, таких як миттєві транзакції (InstantSend), підвищена конфіденційність (PrivateSend) та децентралізоване управління (governance).

З точки зору розгортання, базова конфігурація вузла Dash подібна до інших систем, заснованих на коді Bitcoin, і передбачає встановлення та налаштування програмного демона. Проте повноцінна участь у мережі на рівні майстерноди значно ускладнює цей процес і вимагає від адміністратора додаткових технічних знань і навичок.

Однією з ключових вимог є необхідність блокування криптографічної застави (collateral), яка виступає гарантією добросовісної поведінки вузла. Ця застава зберігається на спеціальному гаманці та не може бути використана під час роботи майстерноди, що створює економічний стимул для підтримки стабільності мережі.

Крім того, необхідно здійснити генерацію та налаштування унікального приватного ключа майстерноди (*masternodeprivkey*), який використовується для ідентифікації вузла в мережі та підпису службових повідомлень. Це додає ще один рівень складності, пов'язаний із безпечним зберіганням ключів і запобіганням несанкціонованому доступу.

Важливим аспектом є також вимога до інфраструктури: майстернода повинна працювати на сервері зі статичною IP-адресою, що забезпечує її постійну доступність у мережі. Додатково вузол повинен відповідати вимогам протоколу Proof-of-Service, який контролює якість обслуговування мережі та перевіряє, чи виконує майстернода свої функції належним чином. Таким чином, архітектура Dash поєднує в собі елементи класичної монолітної моделі (на рівні майнінгу) та модульного підходу (через виділення окремого рівня майстернод). Це робить систему більш функціональною та гнучкою порівняно з Bitcoin, але водночас ускладнює процес розгортання та адміністрування.

У підсумку, Dash демонструє перехідний етап розвитку блокчейн-систем — від простих монолітних рішень до більш складних багаторівневих архітектур, що дозволяють розширювати функціональність мережі без кардинальної зміни її базових принципів.

1.4. NEO: Гетерогенне середовище виконання

Платформа NEO займає особливе місце серед сучасних блокчейн-систем, оскільки вона орієнтована не лише на реалізацію криптовалюти, а й на створення повноцінної інфраструктури для цифрової економіки та виконання смарт-контрактів. За своєю концепцією NEO часто розглядається як альтернатива Ethereum, проте її архітектура має суттєві відмінності як у підходах до консенсусу, так і в організації середовища виконання програм.

Однією з ключових особливостей NEO є використання алгоритму консенсусу dBFT (Delegated Byzantine Fault Tolerance). На відміну від Proof-of-Work або Proof-of-Stake систем, цей механізм базується на делегованій моделі прийняття рішень, у якій спеціально обрані вузли відповідають за досягнення згоди в мережі. Такий підхід дозволяє значно підвищити швидкість підтвердження транзакцій і забезпечити високу пропускну здатність системи, однак водночас вимагає довіри до обмеженого кола учасників, що частково зменшує рівень децентралізації.

Другим важливим компонентом архітектури є власна віртуальна машина NeoVM, яка відповідає за виконання смарт-контрактів. Вона відрізняється від

віртуальної машини Ethereum (EVM) тим, що орієнтована на підтримку кількох мов програмування та більш гнучку інтеграцію з традиційними програмними екосистемами. Саме це робить NEO гетерогенним середовищем виконання, у якому розробка смарт-контрактів не обмежується однією спеціалізованою мовою.

З точки зору розгортання та конфігурації вузла, основним інструментом взаємодії з мережею є консольний інтерфейс Neo-CLI. Через нього здійснюється налаштування вузла, синхронізація з мережею, а також управління базовими параметрами блокчейну. На відміну від більш складних багатокomпонентних архітектур, процес запуску вузла NEO є відносно централізованим, проте вимагає специфічних знань щодо структури платформи та її середовища виконання.

Головною особливістю NEO з точки зору розробки є підхід до смарт-контрактів. Замість використання єдиної вузькоспеціалізованої мови програмування, як це реалізовано в Ethereum (наприклад, Solidity), система дозволяє використовувати традиційні мови програмування, такі як C#, Python або Java. Однак для цього необхідно налаштувати відповідні компілятори та інструменти, які транслюють вихідний код у байткод, сумісний із NeoVM.

Таким чином, адміністрування та розробка в екосистемі NEO зміщується від вивчення спеціалізованих блокчейн-мов до налаштування середовища інтеграції класичних мов програмування. Це робить платформу більш доступною для традиційних розробників, але водночас додає складності на етапі конфігурації інструментального середовища.

У підсумку, NEO демонструє альтернативний підхід до побудови блокчейн-систем, де акцент зроблено на гнучкість середовища виконання та інтеграцію з існуючими програмними технологіями, що відрізняє її як від класичних монолітних систем, так і від більш поширених модульних архітектур.

Розділ 2. Технічні особливості розгортання блокчейн-систем. Загальна системна декомпозиція криптовалютних систем.

1. Загальний підхід системного аналітика до блокчейн-систем

З точки зору системного аналізу, будь-яка блокчейн-платформа розглядається як розподілена інформаційна система, що складається з набору взаємопов'язаних підсистем. При формуванні технічних вимог аналітик не описує лише функції системи, а визначає обмеження, залежності, інфраструктурні вимоги та правила взаємодії модулів.

У випадку блокчейнів ключовими є такі рівні вимог:

- вимоги до зберігання даних (data layer)
- вимоги до консенсусу (consensus layer)
- вимоги до виконання логіки (execution layer)
- вимоги до мережевої взаємодії (network layer)
- вимоги до ініціалізації системи (bootstrap layer)

Саме ці рівні формують архітектурні обмеження, які визначають можливість або неможливість сумісності між різними системами.

2. Вимоги до системи зберігання даних (Data Layer Requirements)

При формуванні вимог до зберігання стану аналітик повинен враховувати модель даних, оскільки вона визначає всю подальшу архітектуру системи.

У Bitcoin, Litecoin та Dash використовується UTXO-модель, яка вимагає реалізації сховища невитрачених виходів транзакцій. У технічному завданні це означає необхідність підтримки швидкого доступу до індексованих записів типу “output → spent/unspent”. База даних у таких системах оптимізується під операції читання і валідації транзакцій, а не під зберігання глобального стану.

У NEO та Ethereum використовується account-based модель, яка вимагає збереження глобального стану мережі. Це означає, що система повинна підтримувати постійне оновлення записів типу “account balance + nonce + contract state”. Така вимога суттєво збільшує навантаження на дискову підсистему та потребує використання високопродуктивних SSD і оптимізованих key-value баз даних.

З точки зору системного аналізу, ці дві моделі є несумісними, оскільки вимоги до структури даних, механізму оновлення стану та логіки валідації транзакцій принципово відрізняються.

3. Вимоги до консенсусного рівня (Consensus Layer Requirements)

Консенсусний рівень визначає правила узгодження стану між вузлами мережі, тому при формуванні вимог аналітик повинен враховувати модель довіри та тип алгоритму.

Bitcoin і Litecoin використовують Proof-of-Work, що накладає вимогу на наявність обчислювальних ресурсів для виконання хешування. У технічному завданні це означає необхідність підтримки конкурентного середовища майнерів, де ймовірність створення блоку залежить від обчислювальної потужності.

Dash розширює цю модель за рахунок masternodes, що формує дворівневу систему консенсусу. Це створює додаткову вимогу до наявності економічного

забезпечення вузлів у вигляді застави, а також до постійної доступності сервісних вузлів.

NEO використовує dBFT, що вимагає визначеного набору консенсусних вузлів (bookkeepers), які мають право формувати блоки. Це означає, що система повинна підтримувати механізм вибору делегатів і миттєвої фіналізації блоків.

Ethereum використовує Proof-of-Stake, де ключовою вимогою є наявність механізму стейкінгу та Beacon Chain, яка координує роботу валідаторів. Це створює вимогу до синхронної роботи двох клієнтів — execution і consensus.

Таким чином, консенсусні модулі різних систем не можуть бути взаємозамінними, оскільки вони базуються на різних математичних моделях довіри та різних принципах фіналізації стану.

4. Вимоги до виконувального середовища (Execution Layer Requirements)

При проєктуванні системи аналітик повинен визначити, чи потребує система середовища виконання смарт-контрактів.

Bitcoin, Litecoin та частково Dash не мають повноцінного execution layer. Вони використовують Script — обмежену стекову мову, що виконує лише прості логічні перевірки. У вимогах це означає відсутність необхідності у віртуальній машині, але також обмежує можливість розширення функціоналу системи.

NEO та Ethereum, навпаки, вимагають наявності віртуальної машини. Ethereum Virtual Machine потребує реалізації газової моделі обчислень, ізоляції виконання коду та підтримки Turing-complete логіки. NeoVM також вимагає середовища виконання байт-коду, але з іншою структурою інструкцій.

З точки зору системного аналізу, ці середовища є несумісними, оскільки:

- різні формати байт-коду
- різні моделі виконання
- різні механізми обробки помилок
- різні правила обліку ресурсів (gas vs відсутність gas)

5. Вимоги до мережевої взаємодії (Network Layer Requirements)

Мережевий рівень визначає, як вузли знаходять один одного та обмінюються даними. У всіх системах використовується P2P-архітектура, однак протоколи взаємодії відрізняються.

Bitcoin і Litecoin використовують DNS seeds, peer exchange та статичні вузли для формування мережі. Dash додає до цього рівня взаємодію з masternodes, що створює додаткові вимоги до стабільності з'єднання.

NEO використовує більш контрольовану мережу консенсусних вузлів, що фактично наближає її до частково централізованої архітектури. Ethereum використовує складну систему discovery, яка включає bootnodes та механізм ідентифікації через chainId.

З точки зору вимог, ключова проблема полягає у відсутності єдиного протоколу handshake. Це означає, що вузли різних систем не можуть встановлювати коректні з'єднання навіть за наявності мережевої доступності.

6. Вимоги до ініціалізації системи (Bootstrap Requirements)

Процес ініціалізації вузла визначає початковий стан системи та правила синхронізації.

У Bitcoin, Litecoin та Dash використовується жорстко закодований genesis-блок, що означає незмінність початкових параметрів системи. Конфігурація здійснюється через .conf файли.

Ethereum використовує genesis.json, який дозволяє гнучко визначати параметри мережі, включаючи gas limit, початкові баланси та версії протоколу. Це значно ускладнює вимоги до валідатора, оскільки система повинна інтерпретувати структурований JSON.

NEO також використовує попередньо визначену емісію та конфігурацію вузлів консенсусу.

З точки зору системного аналізу, різниця у форматах ініціалізації означає повну несумісність bootstrap-модулів між системами.

7. Загальний висновок щодо технічних вимог

Аналіз технічних вимог показує, що кожна блокчейн-система формує власну замкнуту архітектурну модель, яка включає унікальні правила зберігання даних, консенсусу, виконання коду та ініціалізації.

З позиції системного аналітика це означає, що модулі різних систем не можуть бути взаємозамінними, оскільки вони:

- базуються на різних моделях даних (UTXO vs account-based)
- використовують різні консенсусні алгоритми (PoW, PoS, dBFT)
- мають несумісні виконувальні середовища (Script vs EVM vs NeoVM)
- використовують різні протоколи ініціалізації та мережевої взаємодії

Таким чином, кожна система є автономною архітектурною екосистемою, а її модулі можуть бути інтегровані лише на рівні зовнішніх API або міжмережевих мостів, але не на рівні внутрішньої архітектури.

З позиції системного аналітика всі системи можна декомпонувати на модулі:

1. Мережевий модуль (P2P communication)

Забезпечує децентралізовану взаємодію між вузлами мережі без центрального сервера. Відповідає за передачу транзакцій, блоків і синхронізацію даних між учасниками.

2. Модуль консенсусу

Використовується для досягнення згоди між вузлами щодо єдиного стану блокчейну. Він перевіряє блоки та захищає мережу від підробок і подвійних витрат.

3. Транзакційний модуль

Керує створенням, перевіркою та обробкою транзакцій у мережі. Формує пул транзакцій і передає їх для подальшої валідації.

4. Модуль зберігання блокчейну

Відповідає за збереження блоків і транзакцій у базі даних. Також забезпечує швидкий доступ до даних і перевірку цілісності ланцюга.

5. Криптографічний модуль

Забезпечує безпеку даних через цифрові підписи, хешування та шифрування. Використовується для перевірки автентичності транзакцій і захисту від змін.

6. (Опціонально) Виконавчий модуль смарт-контрактів

Виконує смарт-контракти та обробляє їхню логіку в блокчейні. Забезпечує автоматичне виконання умов без участі посередників.

Рівень складності систем (в контексті наведених прикладів вище). Сформуємо порівняльну таблицю розгортання систем

Критерій	Bitcoin	Litecoin	Dash	NEO	Ethereum
Особливості розгортання:	1. запуск full node (Bitcoin Core) 2. синхронізація UTXO ланцюга 3. PoW (SHA-256) - відсутність віртуальної машини - мінімальна конфігурація	1. форк Bitcoin 2. інший алгоритм хешування (Scrypt) 3. швидша генерація блоків	1. двохрівнева мережа: - майнери - masternodes 2. InstantSend (швидкі транзакції) 3. PrivateSend (міксинг транзакцій)	1. account - based модель 2. смарт - контракти 3. підтримка кількох мов програмування 4. dBFT консенсус (delegated Byzantine Fault Tolerance)	1. EVM (Ethereum Virtual Machine) 2. smart contracts як основа системи 3. PoS консенсус 4. account-based модель 5. gas-механізм

Характер:	1. проста інфраструктура 2. висока стабільність 3. низька гнучкість	1. майже ідентичний Bitcoin за архітектурою 2. мінімальні зміни у deployment	1. складніше ніж Bitcoin 2. потребує налаштування masternode-інфраструктури 3. додаткові сервіси приватності	1. потребує runtime середовища для smart contracts 2. складніша інфраструктура ніж UTXO-системи 3. інтеграція з віртуальною машиною	1. найскладніша інфраструктура потребує: 2. execution layer 2. consensus layer 3. RPC API 4. smart contract deployment pipeline
Модель	UTXO	UTXO	UTXO+	Account	Account
Смарт-контракти	ні	ні	обмежено	так	так
Віртуальна машина	ні	ні	ні	так	так (EVM)
Складність deployment	низька	низька	середня	висока	дуже висока
Додаткові сервіси	ні	ні	masternodes	dBFT nodes	execution layer
Масштабованість	низька	низька	середня	висока	висока (L2)

РОЗДІЛ 3. Дослідження можливості взаємозаміни системних модулів

У контексті системної інженерії, під "взаємозаміною модулів" розуміється здатність вилучити певний компонент (наприклад, алгоритм консенсусу або віртуальну машину) з однієї системи та імплементувати його в іншу без повної переробки вихідного коду (plug-and-play).

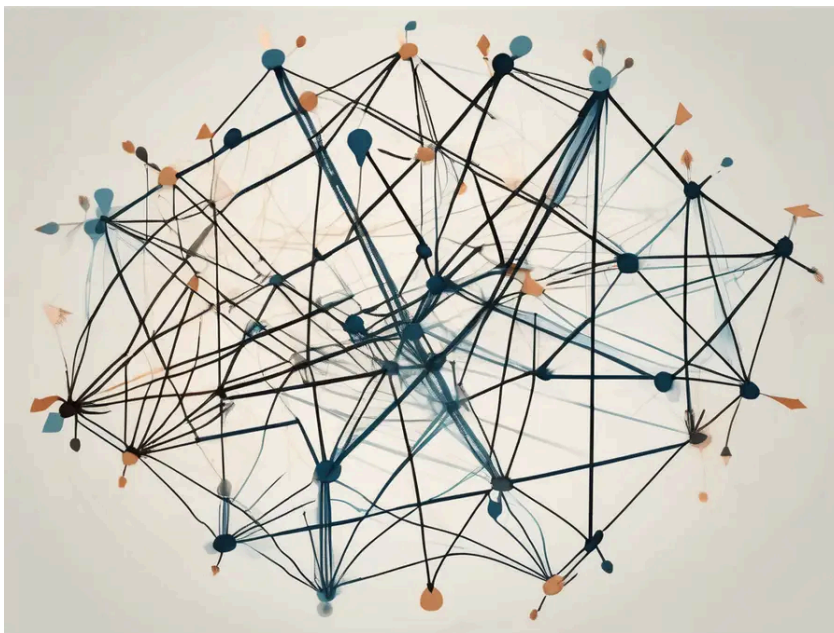
Результат аналізу: Пряма взаємозаміна модулів між системами Ethereum, Bitcoin, Litecoin, Dash та NEO є **фундаментально неможливою**.

Ця неможливість зумовлена трьома критичними факторами несумісності:

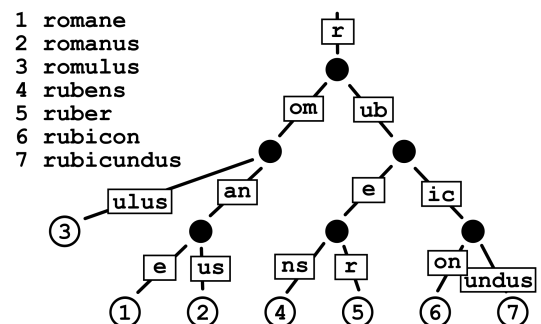
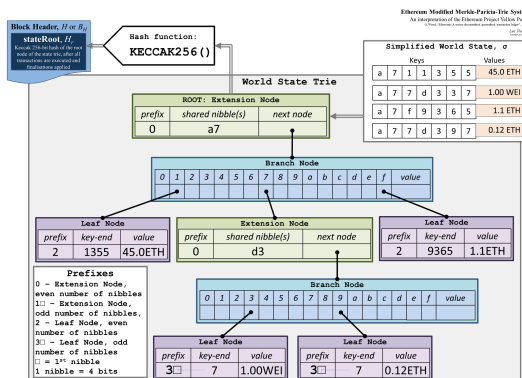
2.1. Несумісність моделей стану (State Models)

Бази даних досліджуваних систем мають різну онтологічну природу:

- **UTXO-модель (Bitcoin, Litecoin, Dash):** Стан мережі представлений у вигляді орієнтованого ациклічного графа невитравлених виходів. Немає поняття "балансу" акаунта, є лише криптографічно заблоковані "монети".



- **Account-based модель (Ethereum, NEO):** Стан зберігається у вигляді криптографічного дерева (Merkle Patricia Trie), де кожен запис містить баланс, попис та стан смарт-контракту. *Висновок:* Модуль виконання Ethereum не здатний прочитати або верифікувати базу даних вузла Bitcoin, оскільки логіка запити балансів несумісна з логікою пошуку вільних входів.



2.2. Несумісність середовищ обчислення (Virtual Machines)

Модулі обробки смарт-контрактів оперують різними наборами інструкцій:

- Ethereum використовує EVM (стекова віртуальна машина, розмір слова 256 біт).
- NEO використовує NeoVM з іншим набором опкодів та архітектурою пам'яті.
- Bitcoin/Litecoin використовують обмежену, не тьюрінг-повну скриптову мову (Script), яка не має стану і використовується виключно для перевірки цифрових підписів. *Висновок:* Спроба інтегрувати модуль EVM у вузол Litecoin або Dash вимагала б повного руйнування базового протоколу консенсусу цих мереж.

2.3. Криптографічна та мережева ізоляція

Вузли взаємодіють між собою через P2P-протоколи, які серіалізують дані по-різному. Ethereum використовує формат RLP (Recursive Length Prefix), тоді як Bitcoin-подібні системи використовують власний бінарний формат серіалізації. Крім того, відрізняються базові хеш-функції (Кессак-256 в Ethereum, SHA-256d в Bitcoin, Scrypt в Litecoin), що робить неможливим використання спільного модуля криптографічної валідації.

ВИСНОВКИ

1. **Диференціація вимог до розгортання:** Налаштування вузлів криптовалютних систем вимагає гетерогенного набору компетенцій. Якщо для Bitcoin та Litecoin достатньо базового адміністрування монолітних демонів, то для розгортання Ethereum та NEO необхідні розширені навички налаштування віртуальних машин, RPC-шлюзів та комунікації між розділеними клієнтами (Execution + Consensus). Налаштування Dash додатково вимагає адміністрування інфраструктури майстернод.
2. **Неможливість модульної інтеграції:** Доведено, що пряма взаємозаміна модулів (таких як середовище виконання чи рівень консенсусу) між досліджуваними системами (Ethereum, Bitcoin, Dash, NEO, Litecoin) є неможливою.
3. **Причини архітектурної несумісності:** Головними бар'єрами є глибока розбіжність структур даних (UTXO проти моделі Акаунтів), несумісність опкодів віртуальних машин (або їх відсутність), а також жорстка прив'язка криптографічних примітивів і мережевих P2P-протоколів до ядра кожної окремої системи.
4. **Перспективи розвитку:** Інтеграція між подібними системами сьогодні можлива не шляхом взаємозаміни модулів їхнього ядра (Layer 1), а виключно через створення надбудов: крос-чейн мостів (Bridges), рішень другого рівня (Layer 2) або шляхом імітації середовища виконання (наприклад, створення EVM-сумісних сайдчейнів).

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Bitcoin: A Peer-to-Peer Electronic Cash System / Satoshi Nakamoto. – 2008.
Офіційний whitepaper, що описує архітектуру Bitcoin, UTXO-модель та Proof-of-Work.
2. Ethereum Whitepaper / Vitalik Buterin. – 2013.
Опис архітектури Ethereum, EVM, account-based моделі та смарт-контрактів.
3. Litecoin Documentation. – Режим доступу: <https://litecoin.org>
Офіційна документація Litecoin, опис відмінностей від Bitcoin та алгоритму Script.
4. Dash Whitepaper та Docs. – Режим доступу: <https://docs.dash.org>
Документація щодо masternodes, InstantSend, PrivateSend та дворівневої архітектури.
5. NEO Whitepaper та Documentation. – Режим доступу: <https://neo.org>
Опис NeoVM, dBFT-консенсусу та двотокенової моделі (NEO/GAS).
6. Ethereum Yellow Paper / Gavin Wood. – 2014.
Формальна специфікація Ethereum, включаючи EVM, gas-модель та структуру стану.
7. Antonopoulos A. M.
Mastering Bitcoin. – O'Reilly Media, 2017.
Детальний технічний опис архітектури Bitcoin, мережних протоколів і транзакцій.
8. Antonopoulos A. M., Wood G.
Mastering Ethereum. – O'Reilly Media, 2018.
Пояснення роботи Ethereum, смарт-контрактів, gas та клієнтської архітектури.
9. Crosby M., Nachiappan P., Verma S., Kalyanaraman V.
“Blockchain Technology: Beyond Bitcoin”. – 2016.
Наукова стаття про загальну архітектуру блокчейн-систем.
10. Zheng Z., Xie S., Dai H., Chen X., Wang H.
“An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends”. – IEEE, 2017.
Огляд архітектур блокчейнів та механізмів консенсусу.
11. LevelDB Documentation. – <https://github.com/google/leveldb>
Документація щодо бази даних, яка використовується в Bitcoin та Ethereum.
12. RocksDB Documentation. – <https://rocksdb.org>
Опис оптимізованої key-value бази даних для високонавантажених систем.